

DISTRIBUIÇÕES DE FREQUÊNCIAS - PYTHON

Para organizar uma distribuição de frequências no PYTHON, usaremos os notebooks já disponíveis no Canvas.

Os comandos definidos nos notebooks disponíveis já importam as bibliotecas necessárias para a leitura dos dados e montagem das tabelas de frequências.

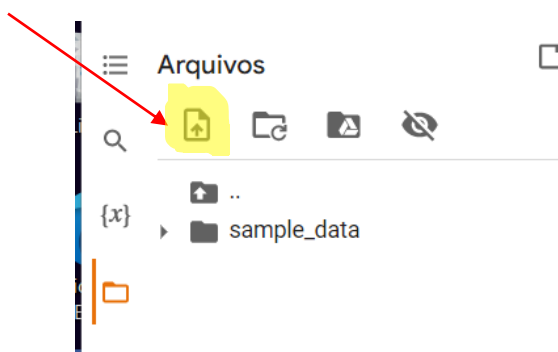
A seguir apresentamos os nomes dos arquivos (.ipynb) que podem ser usados, de acordo com a classificação das variáveis.

Fiquem atentos às informações que devem ser atualizadas em cada notebook (nome do arquivo em Excel, variável a ser considerada) para que os dados sejam lidos corretamente. Após fazer a alteração, é necessário executar cada comando, para que a tabela seja gerada com base no arquivo/coluna escolhidos.

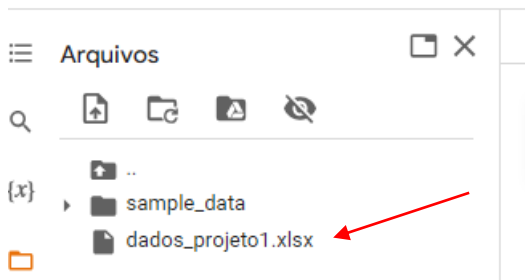
Apresentamos a seguir como fazer as alterações e exibimos alguns dos comandos utilizados nos notebooks. Para ambos, é necessário importar o arquivo do Excel:

➤ Para carregar um arquivo do Excel:

No lado esquerdo, clicar em arquivos e depois na seta, para fazer o upload



Após baixar o arquivo, o nome do arquivo carregado será exibido



Importando as bibliotecas do Python

Para criar tabelas, são necessárias as bibliotecas pandas e math.

1. Importar as bibliotecas Pandas (pd) e math

```
✓ [1] import pandas as pd  
0s import math
```

Leitura do arquivo de dados

Para utilizar o notebook com os seus dados, é necessário alterar o nome do arquivo do Excel, conforme ilustrado abaixo. Após digitar o nome do seu arquivo, o notebook armazena os dados na variável **arquivo** e todos os comandos serão executados com os dados do seu arquivo.

2. Definir a variável "arquivo" pelo nome da planilha.

```
[ ] arquivo='dados_projeto1.xlsx'
```

É necessário definir uma variável para armazenar os dados do arquivo, no exemplo, escolhemos **df**. Ao digitar **df** na linha seguinte, a tabela armazenada é exibida. No exemplo, temos um arquivo com informações sobre Escolaridade, Sexo e Idade.

3. Ler a planilha no Python. Escolha um nome para armazenar. Escolheremos df

```
df=pd.read_excel(arquivo)  
df
```

	Escolaridade	Sexo	Idade
0	superior	fem	21
1	superior	masc	28
2	fundamental	masc	13
3	médio	masc	19
4	médio	fem	19
5	superior	fem	27
6	fundamental	masc	16

Fazendo a contagem da frequência

O comando `value_counts()` faz a contagem (frequência absoluta). É necessário indicar qual a planilha e qual o nome da variável. No exemplo, estão explicitados que os dados se referem à planilha **df** e à variável **Escolaridade**. Os valores serão armazenados em outra planilha. No exemplo, escolhemos **df2**.

4. Agrupando os valores qualitativos a partir da variável selecionada. Escolhemos **Escolaridade** (nome da coluna no arquivo **df**)

```
[4] df2=df.Escolaridade.value_counts()
df2

superior      6
médio         6
fundamental    4
Name: Escolaridade, dtype: int64
```

Podemos ordenar os nomes em ordem alfabética, a partir do comando `sort_index()`

5. Ordenando os valores da variável em ordem alfabética

```
[ ] df2=df.Escolaridade.value_counts().sort_index()
df2

fundamental    4
médio         6
superior       6
Name: Escolaridade, dtype: int64
```

Para obter a frequência relativa, precisamos do total de elementos. Esta soma é feita com o comando `sum()`

6. Contando o total de elementos

```
totaldf2=df2.sum()
totaldf2
```

16

+

Para montar a tabela de distribuição de frequências, precisamos criar um **DataFrame** para a variável desejada. No exemplo, nomeamos essa tabela como **dadosvquali**

7. Montando a tabela de frequências da variável

7.1. Criar um **DataFrame** para a coluna desejada (**dadosvquali** no exemplo)

```
[ ] dadosvquali=pd.DataFrame(df2)
```

```
dadosvquali
```

Escolaridade	
fundamental	4
médio	6
superior	6

Para calcular a frequência relativa, acrescentamos uma nova coluna na tabela ***dadosvquali*** e atribuímos a cada linha o valor da $(\text{frequência absoluta}/\text{total}) \times 100$, arredondando o resultado com 2 casas decimais

7.2 Calcular a frequência relativa em % com 2 casas decimais

```
dadosvquali['Frequência %']=round(dadosvquali['Escolaridade']*100/totaldf,2)
dadosvquali
```

	Escolaridade	Frequência %
fundamental	4	25.0
médio	6	37.5
superior	6	37.5

Agora só falta organizar a nomenclatura da colunas. Observe que na imagem acima, a identificação das colunas está deslocada. O comando DataFrame organiza as informações. Devemos informar o nome da coluna que deve ser exibido e a origem da informação, de acordo com as variáveis armazenadas nos comandos anteriores.

7.3 Criando uma nova tabela, renomeando as colunas

Loading...

```
tabela=pd.DataFrame({"Escolaridade":dadosvquali.index,"Frequência":dadosvquali['Escolarida']})

[ ] tabela
```

	Escolaridade	Frequência	Frequência %
fundamental	fundamental	4	25.0
médio	médio	6	37.5
superior	superior	6	37.5

➤ DADOS QUANTITATIVOS NÃO AGRUPADOS (VARIÁVEL DISCRETA):

NOTEBOOK: **Tab_vardiscreta.ipynb**

ATENÇÃO: USAR ESSA OPÇÃO QUANDO A QUANTIDADE DE VALORES OBSERVADOS DA VARIÁVEL NÃO FOR GRANDE. NESSE CASO, CADA VALOR CORRESPONDE À UMA CLASSE.

Importando as bibliotecas do Python

Para criar tabelas, são necessárias as bibliotecas pandas e math.

1. Importar as bibliotecas Pandas (pd) e math

```
✓ [1] import pandas as pd  
0s import math
```

Leitura do arquivo de dados

Para utilizar o notebook com os seus dados, é necessário alterar o nome do arquivo do Excel, conforme ilustrado abaixo. Após digitar o nome do seu arquivo, o notebook armazena os dados na variável *arquivo* e todos os comandos serão executados com os dados do seu arquivo.

2. Definir a variável "arquivo" pelo nome da planilha.

```
[ ] arquivo='dados_projeto1.xlsx'
```

É necessário definir uma variável para armazenar os dados do arquivo, no exemplo, escolhemos **df**. Ao digitar **df** na linha seguinte, a tabela armazenada é exibida. No exemplo, temos um arquivo com informações sobre Escolaridade, Sexo e Idade.

3. Ler a planilha no Python. Escolha um nome para armazenar. Escolheremos df

```
▶ df=pd.read_excel(arquivo)  
df
```

	Escolaridade	Sexo	Idade
0	superior	fem	21
1	superior	masc	28
2	fundamental	masc	13
3	médio	masc	19
4	médio	fem	19
5	superior	fem	27
6	fundamental	masc	16

Fazendo a contagem da frequência

O comando `value_counts()` faz a contagem (frequência absoluta). É necessário indicar qual a planilha e qual o nome da variável. No exemplo, estão explicitados que os dados referem-se à planilha **df** e à variável **idade**. Os valores serão armazenados em outra planilha. No exemplo, escolhemos **df1**.

Organizando os valores quantitativos a partir da idade

```
df1=df.Idade.value_counts()  
df1
```

```
19    3  
23    3  
22    2  
21    1  
28    1  
13    1  
27    1  
16    1  
15    1  
20    1  
18    1  
Name: Idade, dtype: int64
```

Observe que na tabela acima, os valores da variável (idade) não estão ordenados. Essa ordenação pode ser feita com o comando `sort_index()`, conforme ilustrado abaixo.

5. Ordenando os valores da variável

```
[ ] df1=df.Idade.value_counts().sort_index()
```

```
[ ] df1
```

```
13    1  
15    1  
16    1  
18    1  
19    3  
20    1  
21    1
```

Para obter a frequência relativa, precisamos do total de elementos. Esta soma é feita com o comando `sum()`

6. Contando o total de elementos

```
[ ] totaldf1=df1.sum()  
totaldf1
```

16

Para exibir os dados no formato de uma tabela, criamos uma nova variável para a tabela formatada (no exemplo, **dadosvquant**) e usamos o comando DataFrame, indicando a planilha na qual os dados estão armazenados, **df1**.

7. Montando a tabela de frequências da variável quantitativa discreta

7.1. Criar um DataFrame para a coluna desejada (dadosvquant no exemplo)

```
[ ] dadosvquant=pd.DataFrame(df1)
```

dadosvquant

	Idade
13	1
15	1
16	1
18	1
19	3
20	1
21	1
22	2
23	3
27	1
28	1

nível: 70 71 GR

Para calcular a frequência relativa, acrescentamos uma nova coluna na tabela dadosvquant e atribuímos a cada linha o valor da $(\text{frequência absoluta}/\text{total}) \times 100$, arredondando o resultado com 2 casas decimais

7.2 Calcular a frequência relativa em % com 2 casas decimais

```
[ ] dadosvquant['Frequência %']=round(dadosvquant['Idade']*100/totaldf1,2)
dadosvquant
```

	Idade	Frequência %
13	1	6.25
15	1	6.25
16	1	6.25
18	1	6.25
19	3	18.75
20	1	6.25
21	1	6.25
22	2	12.50
23	3	18.75
27	1	6.25
28	1	6.25

Agora só falta organizar a nomenclatura das colunas. Observe que na imagem acima, a identificação das colunas está deslocada. O comando DataFrame organiza as informações. Devemos informar o nome da coluna que deve ser exibido e a origem da informação, de acordo com as variáveis armazenadas nos comandos anteriores.

7.3 Criando uma nova tabela, renomeando as colunas

```
[ ] tabela=pd.DataFrame({"Idade":dadosvquantil.index,"Frequência":dadosvquantil['Idade'], "Frequência %":dadosvquantil['Frequência %']})
```

tabela

	Idade	Frequência	Frequência %
13	13	1	6.25
15	15	1	6.25
16	16	1	6.25
18	18	1	6.25
19	19	3	18.75
20	20	1	6.25
21	21	1	6.25
22	22	2	12.50
23	23	3	18.75
27	27	1	6.25
28	28	1	6.25

Obs: no notebook estão apresentados os comandos para montar as colunas das frequências acumuladas. Verifique!!

É possível exportar a tabela criada para o Excel, a partir do comando tabela.to_excel

Caso queira enviar essa tabela para o Excel faça

tabela.to_excel("nome do arquivo.xlsx",sheet_name="nome da planilha",index=False)

```
[ ] # criaremos um arquivo de nome Tabela e uma planilha com nome Freq Idade
tabela.to_excel("Tabela.xlsx",sheet_name="Freq Idade",index=False)
```


➤ DADOS QUANTITATIVOS AGRUPADOS (VARIÁVEL CONTÍNUA) :

ATENÇÃO: USAR ESSA OPÇÃO QUANDO A QUANTIDADE DE VALORES OBSERVADOS PARA A VARIÁVEL DISCRETA FOR GRANDE. NESSE CASO, AS CLASSES SERÃO DADAS POR INTERVALOS

Há um comando bem simples para montar a distribuição de frequências por classes:

```
df.variavel.value_counts(bins=k).sort_index()
```

onde **df** é o nome da planilha

variável é o nome da coluna no arquivo

k = número de classes

Com este comando, não é possível alterar os limites de classe. Em algumas situações isso é necessário para melhorar a visualização dos dados, **escolhendo limites de classe com valores inteiros, por exemplo (melhor visualização).**

No notebook abaixo é possível fazer essas alterações.

Notebook: [Tab_varcontinua_classes.ipynb](#)

Leitura do arquivo de dados

Para utilizar o notebook com os seus dados, é necessário alterar os nomes do arquivo do Excel e da coluna da variável, conforme ilustrado abaixo:

2. Defina a variável "arquivo" pelo nome do arquivo da planilha. Defina também a variável "coluna". Essa variável definirá qual a variável será considerada para a construção da tabela.

```
[2] arquivo = 'exemplo tabela.xlsx'
     coluna = 'Sodium (mg)'
```

Ou seja, a variável **arquivo** armazenará os dados do seu arquivo do Excel. E a variável **coluna** armazenará a informação da variável que será considerada.

Para cada variável, é necessário rodar um novo notebook, definindo a coluna desejada.

No notebook, os dados estão armazenados na planilha df.

ESCOLHENDO OS LIMITES DE CLASSE E OBTENDO A FREQUÊNCIA ABSOLUTA

Inicialmente precisamos identificar o valor da menor e da maior observação que constam no conjunto de dados, para garantir que os mesmos estejam incluídos nas classes escolhidas.

```
valormin=df[coluna].min()
valormax=df[coluna].max()
print (valormin,valormax)
```

0 239949

Ou seja, o menor valor observado é 0 e o maior é 239949.

Definindo as classes e calculando a frequência absoluta (**note que o número de classes não deve ser muito grande pois o objetivo da tabela é auxiliar a organização e apresentação dos dados. Em geral deixar de 10-12 linhas no máximo**)). A última classe precisa ter o limite maior que o valor máximo (lembrem que o Python considera intervalo aberto à direita)

```
[5] classes = [0,30000,60000,90000,120000, 150000,180000,210000,240000]
     frequencia = pd.value_counts(pd.cut(x=df[coluna],bins=classes,include_lowest=True,right=False))
     frequencia.sort_index(ascending=True)
```

Incluindo a frequência relativa:

```
Freqpercentual = round(pd.value_counts(pd.cut(x=df[coluna],bins=classes,include_lowest=True,right=False),normalize=True)*100,2)
Freqpercentual.sort_index(ascending=True)
```

Incluindo a frequência acumulada (absoluta e percentual). É necessário ordenar as classes inicialmente.

```
freqpercacum=np.cumsum(Freqpercentual.sort_index())
freqpercacum
```

```
dist_freq = pd.DataFrame({'Frequência': frequencia,'Frequência Relativa (%)':Freqpercentual,'Freq.Acum.': freqacum,'Freq.Rel.Acum. (%)':freqpercacum})
print(dist_freq)
print(f'\n      Variável:{coluna}')
print('      *Limites superiores do intervalo de classes não incluídos')
```

Alguns dos comandos utilizados nos notebooks disponibilizados:

1) Arredondamento de números:

1.1 Arredondamento usando o critério usual: `round(valor)`

```
round(6.9)
```

[x] 7

1.2 Arredondamento para o menor inteiro: `np.floor(valor)`

```
np.floor(6.7)
```

6.0

OBS: O comando só funciona após importarmos a biblioteca numpy (referida no comando acima como np)

➔ IMPORTANTE:

Devido ao sistema de armazenamento do Python, alguns resultados que usam a biblioteca numpy são apresentados no formato:

Números inteiros: `np.int64(resultado)`

Números reais: `np.float(resultado)`